

K-Nearest Neighbors Algorithm

Aim

To implement a K-Nearest Neighbors (KNN) classifier in Python to classify data points based on training and testing datasets.

Procedure

1. Import the necessary libraries such as pandas, numpy, and sklearn.
2. Load the dataset into the program and examine it to understand the features and target variable.
3. Select the relevant independent variable(s) and the dependent variable for classification.
4. Split the dataset into training and testing sets using train-test split to evaluate the model.
5. Create a KNN classifier using sklearn and choose the number of neighbors (k).
6. Train the KNN model on the training dataset and use it to predict the labels for the test dataset.
7. Generate a confusion matrix to observe the number of correct and incorrect predictions for each class.
8. Generate a classification report to evaluate performance metrics such as precision, recall, and F1-score for each class.
9. Analyze the predicted values, confusion matrix, and classification report to assess the effectiveness and accuracy of the KNN model.

```
import pandas as pd
import numpy as np
```

```
df=pd.read_csv('/Iris.csv')
df.head()
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

```
df.isnull().sum()
```

```

      0
sepal.length  0
sepal.width   0
petal.length  0
petal.width   0
variety       0

dtype: int64
```

```
df.variety.value_counts()
```

```

      count
variety
Setosa     50
Versicolor 50
Virginica  50

dtype: int64
```

```
features=df.iloc[:, :-1].values
label=df.iloc[:, 4].values
```

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=42)
```

```
KNN_model=KNeighborsClassifier(n_neighbors=5)
KNN_model.fit(x_train,y_train)
```

```

KNeighborsClassifier
KNeighborsClassifier()

```

```
print(KNN_model.score(x_test,y_test))
print(KNN_model.score(x_train,y_train))
```

```
1.0
0.9666666666666667
```

```
from sklearn.metrics import confusion_matrix
confusion_matrix(label,KNN_model.predict(features))
```

```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])
```

```
from sklearn.metrics import classification_report
print(classification_report(label,KNN_model.predict(features)))
```

	precision	recall	f1-score	support
Setosa	1.00	1.00	1.00	50
Versicolor	0.98	0.94	0.96	50
Virginica	0.94	0.98	0.96	50
accuracy			0.97	150
macro avg	0.97	0.97	0.97	150
weighted avg	0.97	0.97	0.97	150

Result:

Hence, a Python program to implement K-Nearest Neighbors classification has been executed successfully, classifying the data points accurately and evaluating the model using a confusion matrix and classification report.